

UNIT-I:

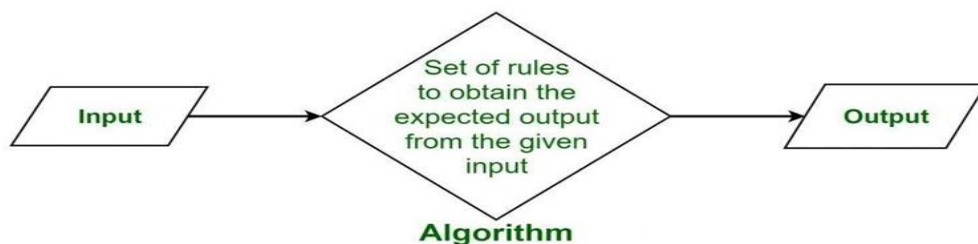
Introduction to Algorithm Analysis, Space and Time Complexity analysis, Asymptotic Notations.

AVL Trees – Creation, Insertion, Deletion operations and Applications

Heap Trees (Priority Queues) – Min and Max Heaps, Operations and Applications

INTRODUCTION TO ALGORITHMS

The word **Algorithm** means “a **process or set of rules** to be followed in calculations or other **problem-solving operations**”. Therefore Algorithm refers to a set of rules/instructions that step-by-step define how a work is to be executed upon in order to get the expected results.

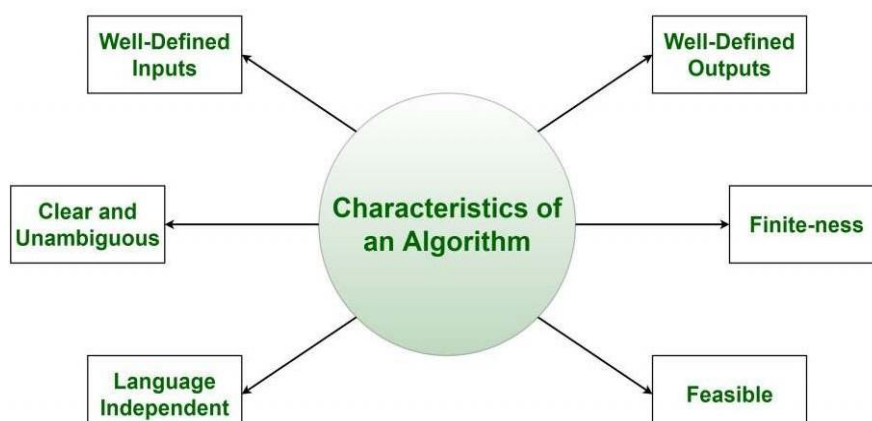


It can be understood by taking an example of cooking a new recipe. To cook a new recipe, one reads the instructions and steps and execute them one by one, in the given sequence. The result thus obtained is the new dish cooked perfectly. Similarly, algorithms help to do a task in programming to get the expected output.

The **Algorithm designed are language-independent**, i.e. they are just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.

Characteristics of an Algorithm:

Characteristics of an Algorithm



- **Clear and Unambiguous:** Algorithm **should be clear** and unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs:** If an algorithm says to take inputs, it **should be well-defined inputs**.
- **Well-Defined Outputs:** The algorithm must clearly define what **output will be**

yielded and it should be well-defined as well.

- **Finite-ness:** The algorithm must be finite, i.e. it should not end up in an infinite loops or similar.
- **Feasible:** The algorithm must be simple, generic and practical, such that it can be executed upon will the available resources. It must not contain some future technology, or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be same, as expected.

Advantages of Algorithms:

- It is easy to understand.
- Algorithm is a step-wise representation of a solution to a given problem.
- In Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms:

- Writing an algorithm takes a long time so it is time-consuming.
- Branching and looping statements are difficult to show in Algorithms.

In order to write an algorithm, following things are needed as a pre-requisite:

1. The **problem that is to be solved by** this algorithm.
2. The **constraints of the problem** that must be considered while solving the problem.
3. The **input** to be taken to solve the problem.
4. The **output** to be expected when the problem is solved.
5. The **solution to this problem, in the given constraints.**

Then the algorithm is written with the help of above parameters such that it solves the problem.

Example: Consider the example to add three numbers and print the sum.

Step 1: Fulfilling the pre-requisites

As discussed above, in order to write an algorithm, its pre-requisites must be fulfilled.

1. **The problem that is to be solved by this algorithm:** Add 3 numbers and print their sum.
2. **The constraints of the problem that must be considered while solving the problem:** The numbers must contain only digits and no other characters.
3. **The input to be taken to solve the problem:** The three numbers to be added.
4. **The output to be expected when the problem is solved:** The sum of the three numbers taken as the input.
5. **The solution to this problem, in the given constraints:** The solution consists of adding the 3 numbers. It can be done with the help of '+' operator, or bit-wise, or any other method.

Step 2: Designing the algorithm

Algorithm to add 3 numbers and print their sum:

1. START
2. Declare 3 integer variables num1, num2 and num3.
3. Take the three numbers, to be added, as inputs in variables num1, num2, and num3 respectively.
4. Declare an integer variable sum to store the resultant sum of the 3 numbers.
5. Add the 3 numbers and store the result in the variable sum.
6. Print the value of variable sum
7. END

Step 3: Testing the algorithm by implementing it.

Analysis on algorithm:

1. **Priori Analysis:** "Priori" means "before". Hence Priori analysis means checking the algorithm before its implementation. In this, the algorithm is checked when it is written in the form of theoretical steps. This Efficiency of an algorithm is measured by assuming that all other factors, for example, processor speed, are constant and have no effect on the implementation. This is done usually by the algorithm designer. It is in this method, that the Algorithm Complexity is determined.
2. **Posterior Analysis:** "Posterior" means "after". Hence Posterior analysis means checking the algorithm after its implementation. In this, the algorithm is checked by implementing it in any programming language and executing it. This analysis helps to get the actual and real analysis report about correctness, space required, time consumed etc.
 - **Time Factor:** Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
 - **Space Factor:** Space is measured by counting the maximum memory space required by the algorithm.

PERFORMANCE ANALYSIS

Performance Analysis refers to the task of determining the computing time and storage space requirements of an algorithm. It is also known as performance analysis.

The main purpose of algorithm analysis is to design most efficient algorithms. The efficiency of the algorithm mainly depends on two factors

- 1 : Space Complexity
- 2 : Time Complexity

1: Space Complexity: The Space Complexity can be defined as amount of space required by an algorithm to run to completion. By computing space complexity we can analysis the algorithm requires less or more space.

The space needed by an algorithm consists of the following components.

1: The fixed static part that is independent of the characteristics (example number size) of the input and outputs. This part includes the instruction space, space for simple variable and fixed size component variables, space for constants etc..

2: A variable dynamic part, that consists of the space needed by component variable whose size is dependent on the particular problem instance at runtime being solved, the space needed by referenced variables and the recursion stack space.

The space complexity requirement $S(p)$ can be given as: $S(p) = C + Sp$

Where **C** is a constant i.e, fixed part. It denotes amount of space required for inputs, outputs, variables and instructions. And **Sp** is a instance characteristics. This is a variable part amount of space required for problem is depends on particular problem instance.

Consider the examples of algorithms to compute the space complexity

Ex1: Algorithm add(a, b, c)

```
{  
//Problem Description: This algorithm computes the addition of 3 numbers  
//Input: a, b and c are of floating type.  
//Output: The addition is returned.  
return a+b+c;  
}
```

The space requirement for algorithm given in $S(p) = C$

If we assume that a, b and c occupy one word size then total size comes to be 3.

2: Time Complexity: The Time Complexity can be defined as amount of computer time required by an algorithm to run to completion.

They are two types of computing time.

1. compile time
2. run time

For calculating the time complexity, only run time is considered.

It is difficult to compute the time complexity in terms of physically clocked time. Because runtime depends on many factors such as system load, number of other programs running and processor speed.

For avoiding this, we are using frequency count is basically a count denoting number of times of execution of statement.

The method to determine the step count of an algorithm is to build a table in which we list the total number of steps contributed by each statement.

First determine the number of steps per execution(s/e) of the statement and the total number of times (ie., frequency) each statement is executed.

By combining these two quantities, the total contribution of all statements, the step count for the entire algorithm is obtained.

Example1:

Statement	S/E	Frequency	TotalSteps
1AlgorithmSum(a,n)	0	-	0
2.{	0	-	0
3. S=0.0;	1	1	1
4. forl=1tondo	1	N+1	N+1
5. s=s+a[l];	1	N	N
6. returns;	1	1	1
7.}	0	-	0
Total			2N+3

ASYMPTOTIC NOTATIONS

Expressing the complexity in terms of its relationship to know function. This type of Notation is called Asymptotic Notations.

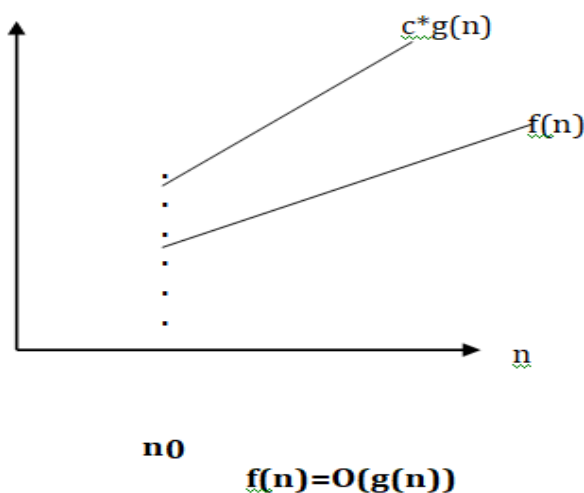
Various types of notations:

- 1 :Big Oh Notation('O')
- 2 :Omega Notation('Ω')
- 3 :Theta Notation('θ')

1: Big Oh Notation ('O'):

Definition: Let $f(n)$ and $g(n)$ be two non-negative functions. $f(n)$ is said to be $O(g(n))$ if and only if there exists positive constants 'c' and 'n₀' such that $f(n) \leq c * g(n)$ for all non-negative values of n. where $n \geq n_0$.

The following graph the curve for Big Oh Notation.



This notation provides an upper bound for the function f i.e, the function $g(n)$ is an upper bound on the value of $f(n)$ for all n , where $n \geq n_0$.

1: Show that $f(n)=8n+128=O(n^2)$.

To Prove.

$$f(n)=8n+128=O(n^2)$$

Proof: If $f(n)=O(g(n))$. We have there exists positive constants ' c ' and ' n_0 ' such that $f(n) \leq c \cdot g(n)$ for all non-negative values of n . where $n \geq n_0$.

Given problem in the form of $f(n)=O(g(n))$.

Here $f(n)=8n+128$

$$g(n)=n^2$$

Now $8n+128 \leq 3 \cdot n^2$

Now N=1.	$8.1+128 \leq 3 \cdot 1$	(false)
Now take N=2	$8.2+128 \leq 3 \cdot 4$	(false)
Now take N=3	$8.3+128 \leq 3 \cdot 9$	(false)
Now take N=4	$8.4+128 \leq 3 \cdot 16$	(false)
Now take N=5	$8.5+128 \leq 3 \cdot 25$	(false)
Now take N=6	$8.6+128 \leq 3 \cdot 36$	(false)
Now take N=7	$8.7+128 \leq 3 \cdot 49$	(false)
Now take N=8	$8.8+128 \leq 3 \cdot 64$	(true)

There exists two positive constants $c=3$, $n_0=8$ such that $8n+128 \leq 3 \cdot n^2$ for all $n \geq 8$.

Therefore $8n+128=O(n^2)$

2: Show that $f(n)=12n^2+6n=O(n^3)$

To Prove.

$$f(n)=12n^2+6n=O(n^3)$$

Proof: If $f(n)=O(g(n))$. We have there exists positive constants ' c ' and ' n_0 ' such that $f(n) \leq c \cdot g(n)$ for all non-negative values of n . where $n \geq n_0$.

Given problem in the form of $f(n)=O(g(n))$.

Here, $f(n)=12n^2+6n$ and $g(n)=n^3$

Now $f(n)=12n^2+6n \leq 4 \cdot n^3$

Now $N=1$. $+6.1 \leq 4 \cdot 1$ (false)

Now $N=2$. $+6.2 \leq 4 \cdot 8$ (false)

Now $N=3$. $+6.3 \leq 4 \cdot 27$ (True)

There exists two positive constants $c=4$, $n_0=3$ such that $12n^2+6n \leq 4 \cdot n^3$ for all $n \geq 3$.

Therefore $12n^2+6n = O(n^3)$

3: consider function $f(n)=2n+2$ and $g(n)=n^2$.

Proof: If $f(n)=O(g(n))$. We have there exists positive constants ' c ' and ' n_0 ' such that $f(n) \leq c \cdot g(n)$ for all non-negative values of n . where $n \geq n_0$.

Given, $f(n)=2n+2$.
 $g(n)=n^2$

Now $f(n)=2n+2 \leq 2 \cdot n^2$

Now Take $N=1$. $2.1+2 \leq 2 \cdot 1$ (false)

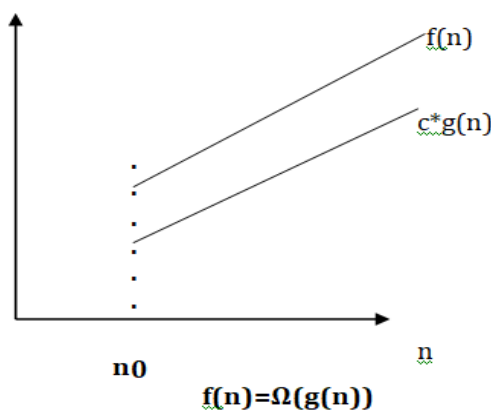
Now Take $N=2$. $2.2+2 \leq 2 \cdot 4$ (True).

There exists two positive constants $c=2$, $n_0=2$ such that $2n+2 \leq 2 \cdot n^2$ for all $n \geq 2$.

2: Omega Notation (Ω):

Definition: Let $f(n)$ and $g(n)$ be two non-negative functions. $f(n)$ is said to be $\Omega(g(n))$ if and only if there exists positive constants ' c ' and ' n_0 ' such that $f(n) \geq c \cdot g(n)$ for all non-negative values of n . where $n \geq n_0$.

The following graph the curve for Omega Notation.



This notation provides an upper bound for the function f i.e, the function $g(n)$ is an lower bound on the value of $f(n)$ for all n , where $n \geq n_0$.

1: Show that $f(n)=8n+128=\Omega(n^2)$. To Prove.

$$f(n)=8n+128=\Omega(n^2)$$

Proof: If $f(n)=O(g(n))$. We have there exists positive constants ' c ' and ' n_0 ' such that $f(n) \geq c \cdot g(n)$ for all non-negative values of n . where $n \geq n_0$.

Given problem in the form of $f(n)=O(g(n))$.

Here $f(n)=8n+128$

$$g(n)=n^2$$

Now $8n+128 \geq 1 \cdot n^2$

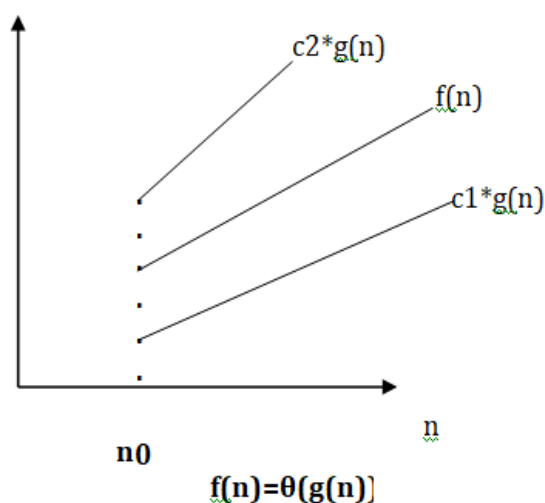
Now $N=1$. $8.1+128 \geq 1 \cdot 1$ (TRUE)

There exists two positive constants $c=1$, $n_0=1$ such that $8n+128 \leq 1 \cdot n^2$ for all $n > 1$.
Therefore $8n+128=O(n^2)$

3: Theta Notation (θ):

Definition: Let $f(n)$ and $g(n)$ be two non-negative functions. $f(n)$ is said to be $\theta(g(n))$ if and only if there exists positive constants ' c_1 ', ' c_2 ' and ' n_0 ' such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all non-negative values of n . where $n \geq n_0$.

The following graph the curve for Theta Notation.



This notation provides both lower bound and upper bound for the function $f(n)$ i.e, $g(n)$ is both lower and upper bound on the value of $f(n)$, for large n .

1: Show that $f(n)=8n+128=\theta(n^2)$. To Prove.

$$f(n)=8n+128=O(n^2)$$

Proof:

If $f(n)=O(g(n))$. We have there exists positive constants '**c1**', '**c2**' and '**n0**'

Such that $c1g(n)\leq f(n)\leq c2g(n)$ for all non-negative values of n. where $n>_n0$.

Given problem in the form of $f(n)=\theta(g(n))$.

$$\begin{aligned}\text{Here} \quad f(n) &= 8n+128 \\ g(n) &= n^2\end{aligned}$$

$$\text{Now,} \quad 1*n^2 \leq 8n+128 \leq 3*n^2$$

$$\text{Now Take } N=1 \quad 1*1 \leq 8.1+128 \leq 3*1 \quad (\text{False})$$

$$\text{Now Take } N=2 \quad 1*4 \leq 8.2+128 \leq 3*4 \quad (\text{False})$$

$$\text{Now Take } N=3 \quad 1*9 \leq 8.3+128 \leq 3*9 \quad (\text{False})$$

$$\text{Now Take } N=4 \quad 1*16 \leq 8.4+128 \leq 3*16 \quad (\text{False})$$

$$\text{Now Take } N=5 \quad 1*25 \leq 8.5+128 \leq 3*25 \quad (\text{False})$$

$$\text{Now Take } N=6 \quad 1*6 \leq 8.6+128 \leq 3*36 \quad (\text{False})$$

$$\text{Now Take } N=7 \quad 1*49 \leq 8.7+128 \leq 3*49 \quad (\text{False})$$

$$\text{Now Take } N=8 \quad 1*64 \leq 64+128 \leq 3*64 \quad (\text{true}).$$

There exists positive constants $c1=1$, $c2=3$, **$n0=8$** such that $1*n^2 \leq 8n+128 \leq 3*n^2$ for all $n \geq 8$

4: Little Oh Notation: The Little Oh Notation is denoted by '**o**'. The function $f(n)=o(g(n))$ iff

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Average of Best Case, Worst Case, Average Case

1: Best Case: If the algorithm takes the minimum amount of time to run to completion for a specific set of inputs is called Best Case Time Complexity.

Example: While search in g a particular element by using sequential search. We get the desired element at starting of the list.

2: Worst Case: If the algorithm takes maximum amount of time to run to completion for a specific set of inputs is called Worst Case Time Complexity.

Example: While searching an element by using linear search. If we get the desired element at end of the list.

3: Average Case: If the algorithm takes average amount of time to run to completion for a specific set of inputs is called Average Case.

Example: Binary Search.

AVL TREE

AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.

AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

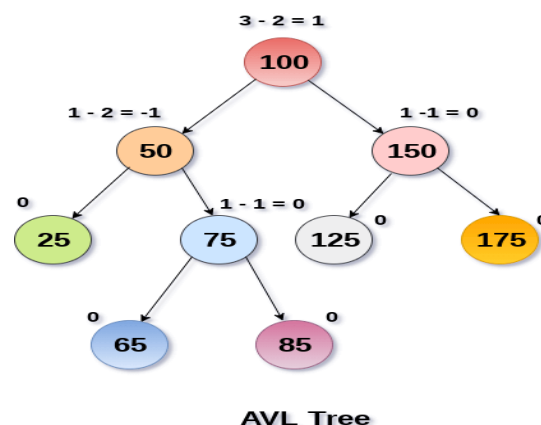
Balance Factor (k) = height (left(k)) - height (right(k))

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



Complexity

Algorithm	Average case	Worst case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

Operations on AVL tree

Due to the fact that, AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree. Searching and traversing do not lead to the violation in property of AVL tree. However, insertion and deletion are the operations which can violate this property and therefore, they need to be revisited.

SN	Operation	Description
1	Insertion	Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.
2	Deletion	Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

AVL tree controls the height of the binary search tree by not letting it to be skewed. The time taken for all operations in a binary search tree of height h is $O(h)$. However, it can be extended to $O(n)$ if the BST becomes skewed (i.e. worst case). By limiting this height to $\log n$, AVL tree imposes an upper bound on each operation to be $O(\log n)$ where n is the number of nodes.

AVL Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:

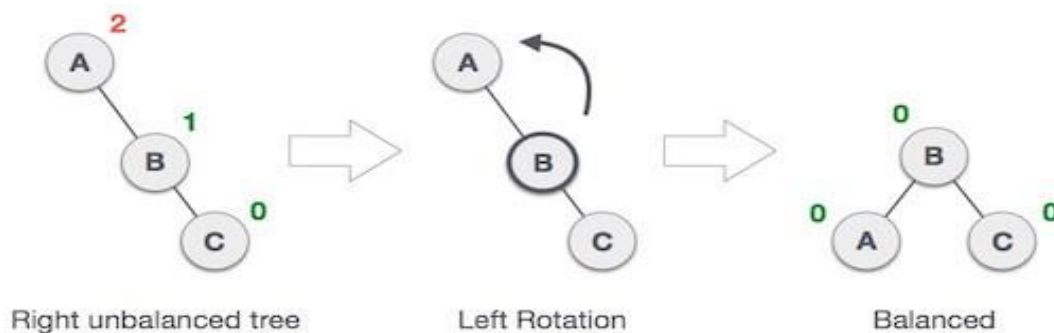
1. L L rotation: Inserted node is in the left subtree of left subtree of A
2. R R rotation : Inserted node is in the right subtree of right subtree of A
3. L R rotation : Inserted node is in the right subtree of left subtree of A
4. R L rotation : Inserted node is in the left subtree of right subtree of A

Where node A is the node whose balance Factor is other than -1, 0, 1.

The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

1. RR Rotation

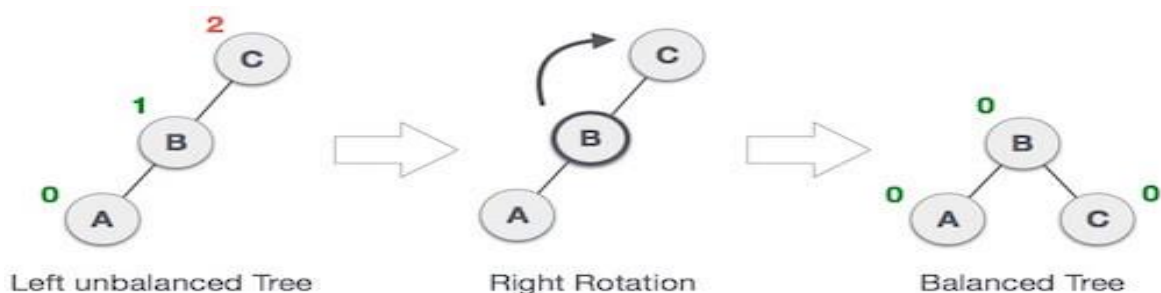
When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, RR rotation is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



In above example, node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. We perform the RR rotation on the edge below A.

2. LL Rotation

When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, LL rotation is clockwise rotation, which is applied on the edge below a node having balance factor 2.



In above example, node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. We perform the LL rotation on the edge below A.

3. LR Rotation

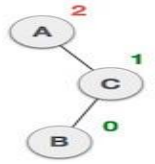
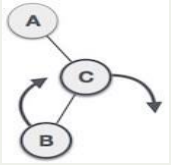
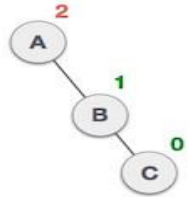
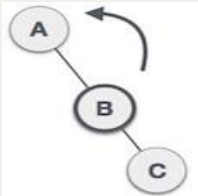
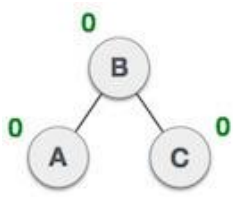
Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

Let us understand each and every step very clearly:

State	Action
	A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C
	As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node A, has become the left subtree of B.
	After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C
	Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B
	Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

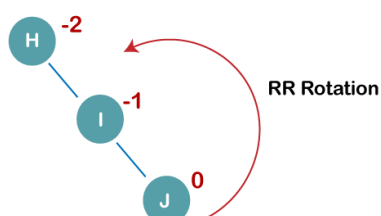
As already discussed, that double rotations are bit tougher than single rotation which has already explained above. R L rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

State	Action
	A node B has been inserted into the left subtree of C the right subtree of A , because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A
	As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at C is performed first. By doing RR rotation, node C has become the right subtree of B .
	After performing LL rotation, node A is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A .
	Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A . node C has now become the right subtree of node B , and node A has become the left subtree of B .
	Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

Q: Construct an AVL tree having the following elements

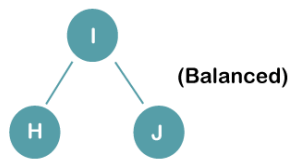
H, I, J, B, A, E, C, F, D, G, K, L

1. Insert H, I, J

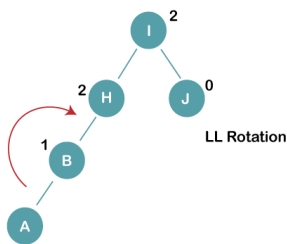


On inserting the above elements, especially in the case of **H**, the BST becomes unbalanced as the Balance Factor of **H** is -2. Since the BST is right-skewed, we will perform RR Rotation on node **H**.

The resultant balance tree is:

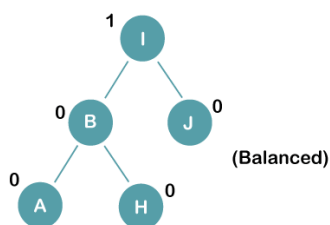


2. Insert B, A

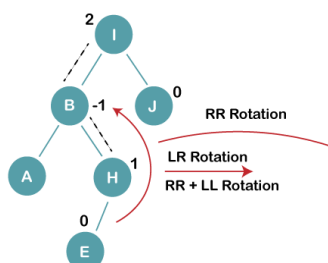


On inserting the above elements, especially in case of A, the BST becomes unbalanced as the Balance Factor of H and I is 2, we consider the first node from the last inserted node i.e. H. Since the BST from H is left-skewed, we will perform LL Rotation on node H.

The resultant balance tree is:



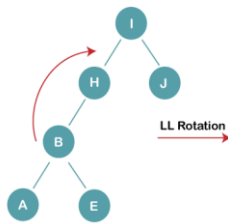
3. Insert E



On inserting E, BST becomes unbalanced as the Balance Factor of I is 2, since if we travel from E to I we find that it is inserted in the left subtree of right subtree of I, we will perform LR Rotation on node I. LR = RR + LL rotation

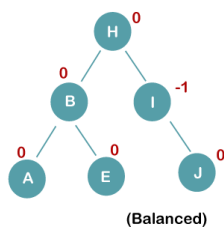
3 a) We first perform RR rotation on node B

The resultant tree after RR rotation is:

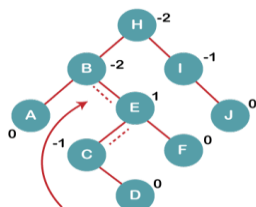


3b) We first perform LL rotation on the node I

The resultant balanced tree after LL rotation is:



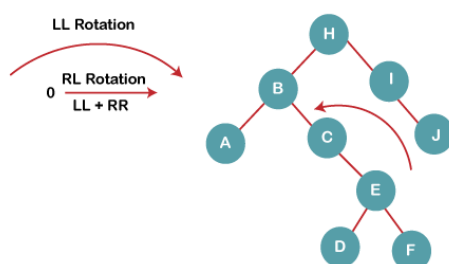
4. Insert C, F, D



On inserting C, F, D, BST becomes unbalanced as the Balance Factor of B and H is -2, since if we travel from D to B we find that it is inserted in the right subtree of left subtree of B, we will perform RL Rotation on node I. RL = LL + RR rotation.

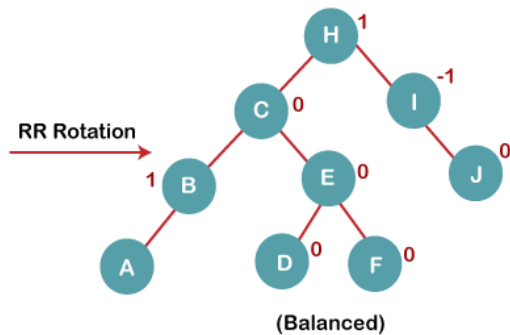
4a) We first perform LL rotation on node E

The resultant tree after LL rotation is:

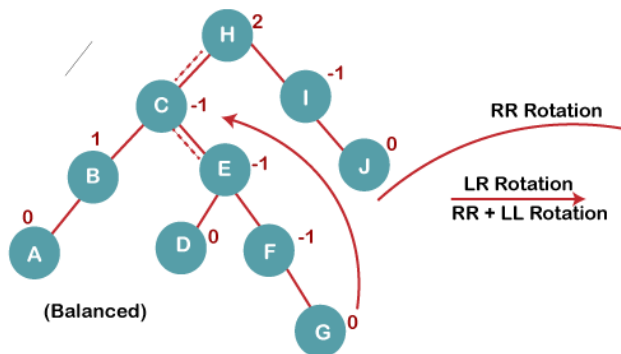


4b) We then perform RR rotation on node B

The resultant balanced tree after RR rotation is:



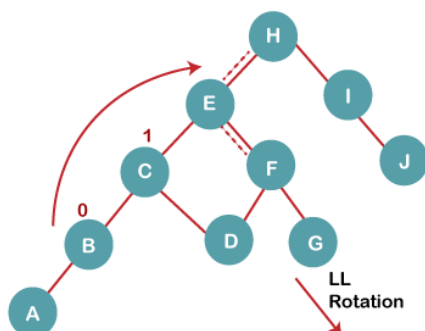
5. Insert G



On inserting G, BST become unbalanced as the Balance Factor of H is 2, since if we travel from G to H, we find that it is inserted in the left subtree of right subtree of H, we will perform LR Rotation on node I. LR = RR + LL rotation.

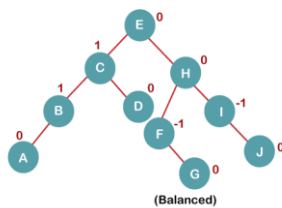
5 a) We first perform RR rotation on node C

The resultant tree after RR rotation is:

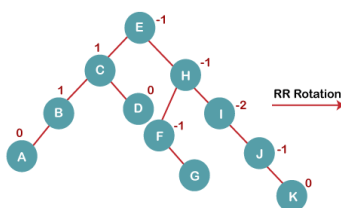


5 b) We then perform LL rotation on node H

The resultant balanced tree after LL rotation is:

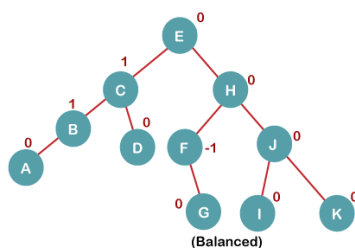


6. Insert K



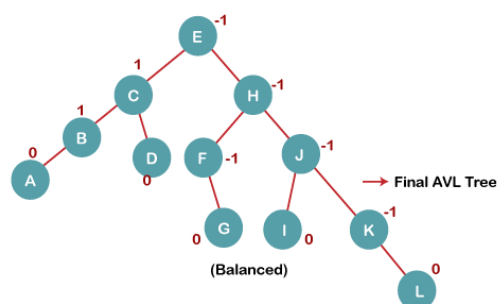
On inserting K, BST becomes unbalanced as the Balance Factor of I is -2. Since the BST is right-skewed from I to K, hence we will perform RR Rotation on the node I.

The resultant balanced tree after RR rotation is:



7. Insert L

On inserting the L tree is still balanced as the Balance Factor of each node is now either, -1, 0, +1. Hence the tree is a Balanced AVL tree



DELETION IN AVL TREE

Deleting a node from an AVL tree is similar to that in a binary search tree. Deletion may disturb the balance factor of an AVL tree and therefore the tree needs to be rebalanced in order to maintain the AVLness. For this purpose, we need to perform rotations. The two types of rotations are L rotation and R rotation. Here, we will discuss R rotations. L rotations are the mirror images of them.

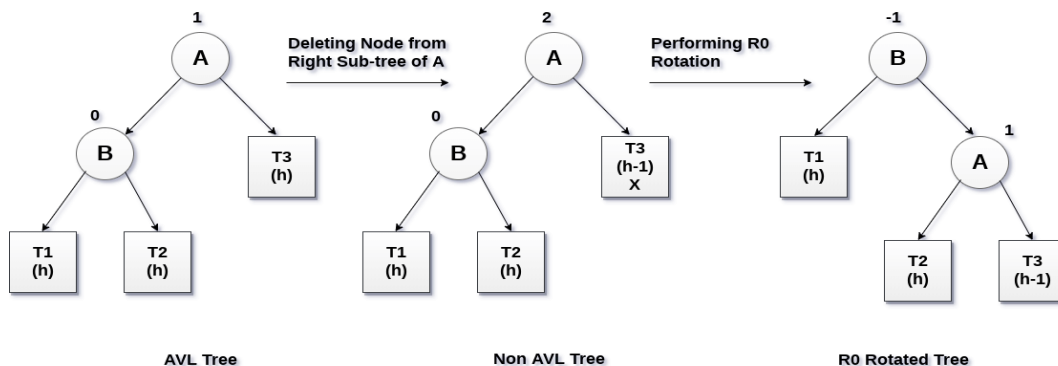
If the node which is to be deleted is present in the left sub-tree of the critical node, then L rotation needs to be applied else if, the node which is to be deleted is present in the right sub-tree of the critical node, the R rotation will be applied.

Let us consider that, A is the critical node and B is the root node of its left sub-tree. If node X, present in the right sub-tree of A, is to be deleted, then there can be three different situations:

R0 rotation (Node B has balance factor 0)

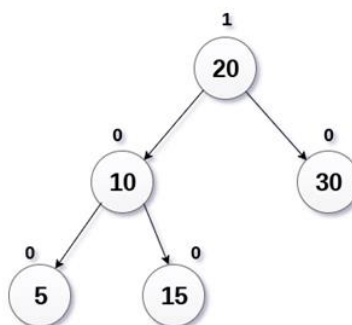
If the node B has 0 balance factor, and the balance factor of node A disturbed upon deleting the node X, then the tree will be rebalanced by rotating tree using R0 rotation.

The critical node A is moved to its right and the node B becomes the root of the tree with T1 as its left sub-tree. The sub-trees T2 and T3 becomes the left and right sub-tree of the node A. the process involved in R0 rotation is shown in the following image.



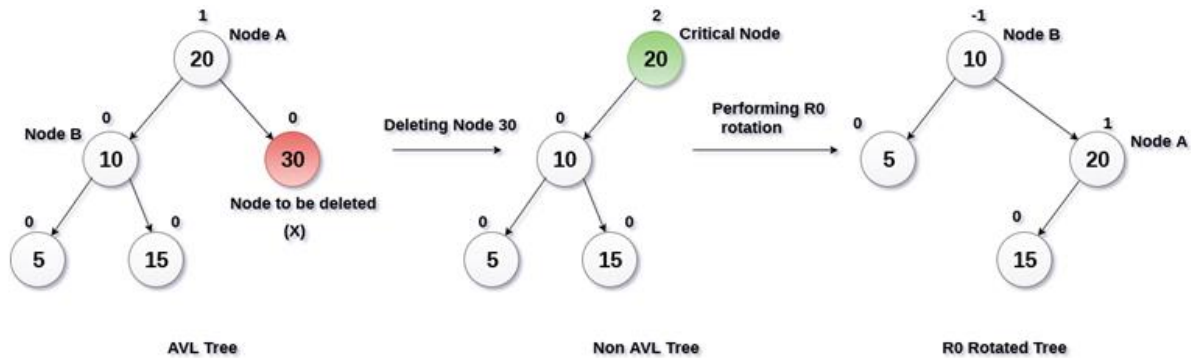
Example:

Delete the node 30 from the AVL tree shown in the following image.



Solution

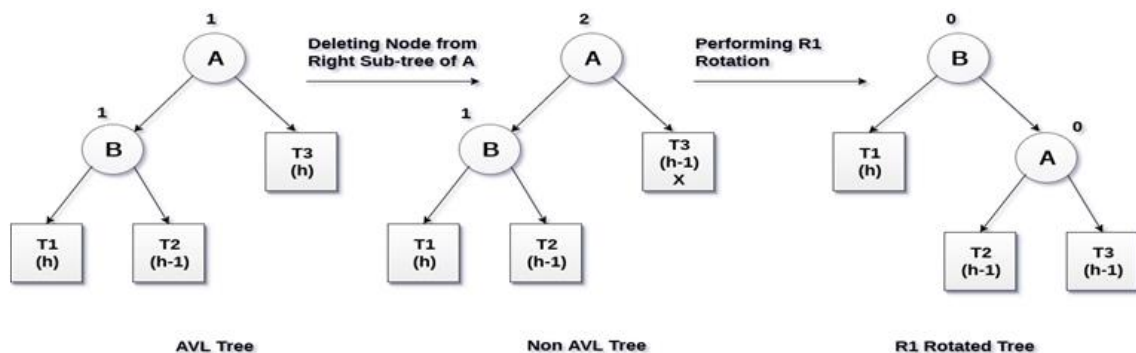
In this case, the node B has balance factor 0, therefore the tree will be rotated by using R0 rotation as shown in the following image. The node B(10) becomes the root, while the node A is moved to its right. The right child of node B will now become the left child of node A.



R1 Rotation (Node B has balance factor 1)

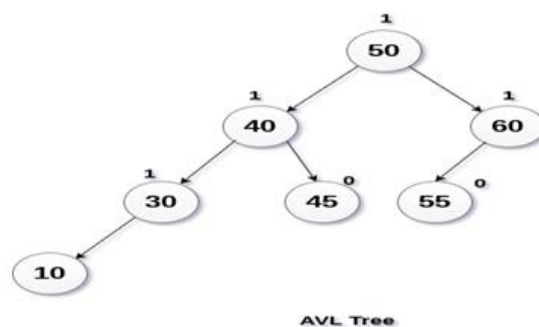
R1 Rotation is to be performed if the balance factor of Node B is 1. In R1 rotation, the critical node A is moved to its right having sub-trees T2 and T3 as its left and right child respectively. T1 is to be placed as the left sub-tree of the node B.

The process involved in R1 rotation is shown in the following image.



Example

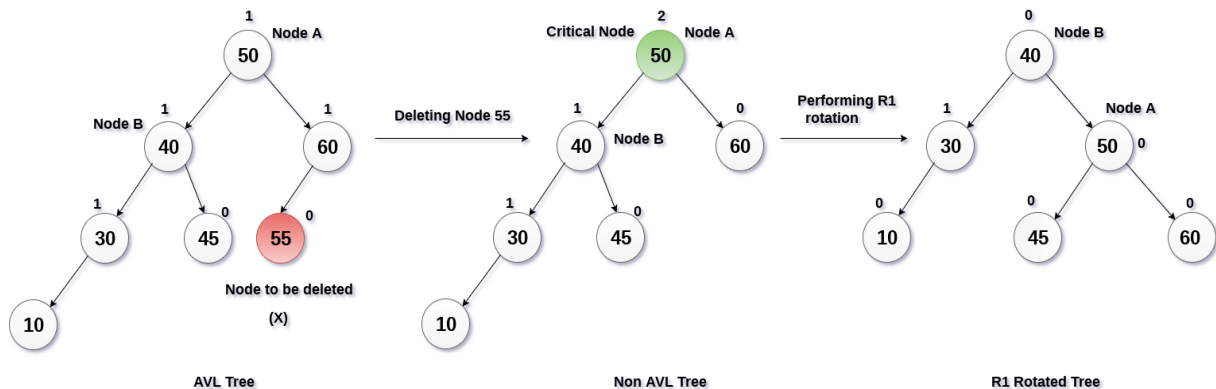
Delete Node 55 from the AVL tree shown in the following image.



Solution :

Deleting 55 from the AVL Tree disturbs the balance factor of the node 50 i.e. node A which becomes the critical node. This is the condition of R1 rotation in which, the node A will be moved to its right (shown in the image below). The right of B is now become the left of A (i.e. 45).

The process involved in the solution is shown in the following image.

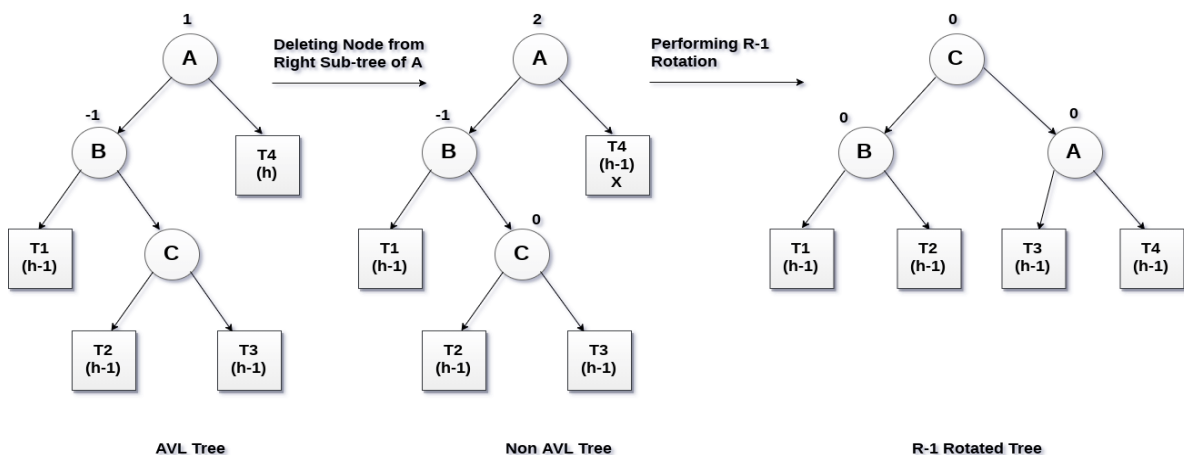


R-1 Rotation (Node B has balance factor -1)

R-1 rotation is to be performed if the node B has balance factor -1. This case is treated in the same way as LR rotation. In this case, the node C, which is the right child of node B, becomes the root node of the tree with B and A as its left and right children respectively.

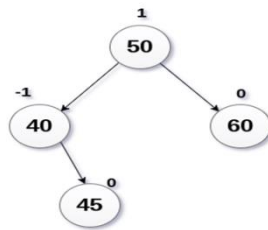
The sub-trees T1, T2 becomes the left and right sub-trees of B whereas, T3, T4 become the left and right sub-trees of A.

The process involved in R-1 rotation is shown in the following image.



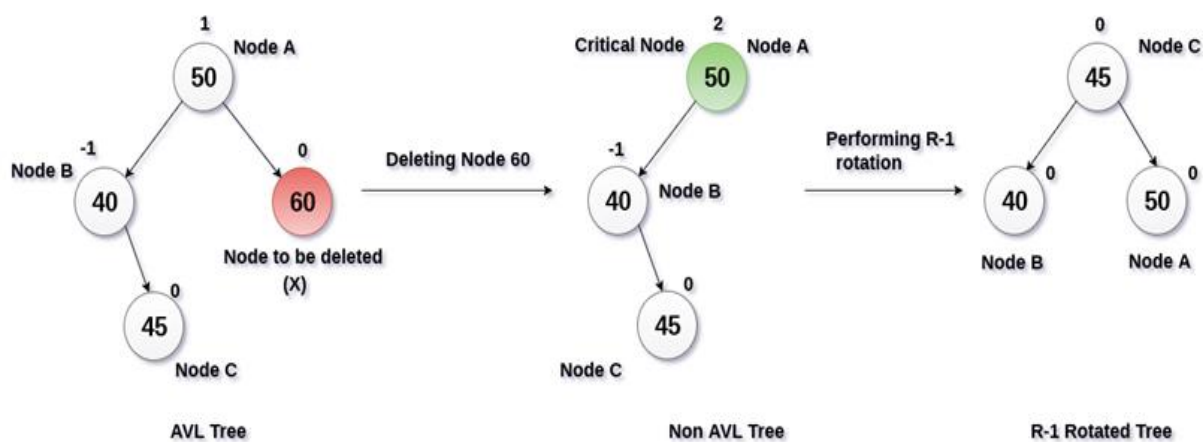
Example

Delete the node 60 from the AVL tree shown in the following image.



Solution:

in this case, node B has balance factor -1. Deleting the node 60, disturbs the balance factor of the node 50 therefore, it needs to be R-1 rotated. The node C i.e. 45 becomes the root of the tree with the node B(40) and A(50) as its left and right child.



APPLICATIONS OF AVL TREES

- Most in-memory sets and dictionaries are stored using AVL trees.
- Database applications, where insertions and deletions are less common but frequent data lookups are necessary, also frequently employ AVL trees.
- In addition to database applications, it is employed in other applications that call for better searching.
- A balanced binary search tree called an AVL tree uses rotation to keep things balanced.
- It may be used in games with plotlines as well.
- It is mostly utilized in business sectors where it is necessary to keep records on the employees that work there and their shift changes.

Heap Trees:

A Heap is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value.

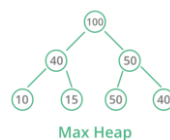
What is Heap Data Structure?

A heap is a binary tree-based data structure that follows the heap property. In a heap, the value of each node is compared to the values of its children in a specific way:

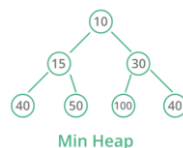
Types of Heaps:

There are **two** main types of heaps:

Max Heap: The root node contains the maximum value, and the values decrease as you move down the tree.



Min Heap: The root node contains the minimum value, and the values increase as you move down the tree.



Operations on Max Heap

The following operations are performed on a Max heap data structure...

1. Finding Maximum
2. Insertion
3. Deletion

Insertion Operation in Max Heap

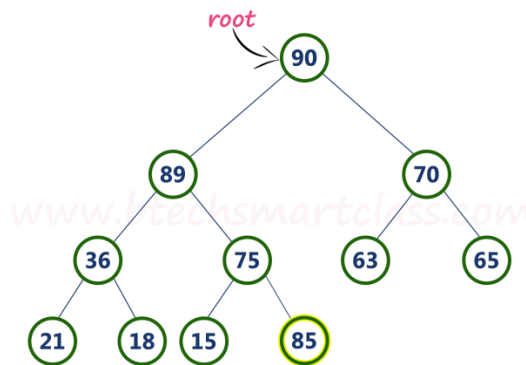
Insertion Operation in max heap is performed as follows...

- **Step 1** - Insert the **newNode** as **last leaf** from left to right.
- **Step 2** - Compare **newNode value** with its **Parent node**.
- **Step 3** - If **newNode value is greater** than its parent, then **swap** both of them.
- **Step 4** - Repeat step 2 and step 3 until newNode value is less than its parent node (or) newNode reaches to root.

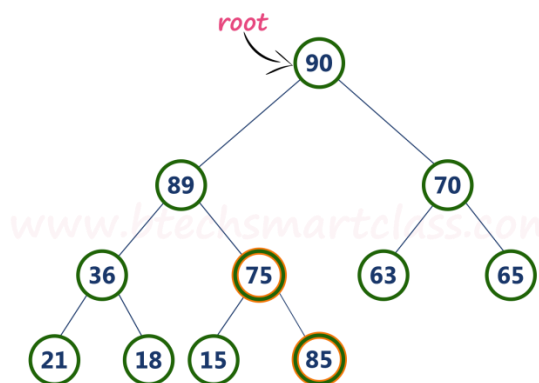
Example

Consider the above max heap. **Insert a new node with value 85.**

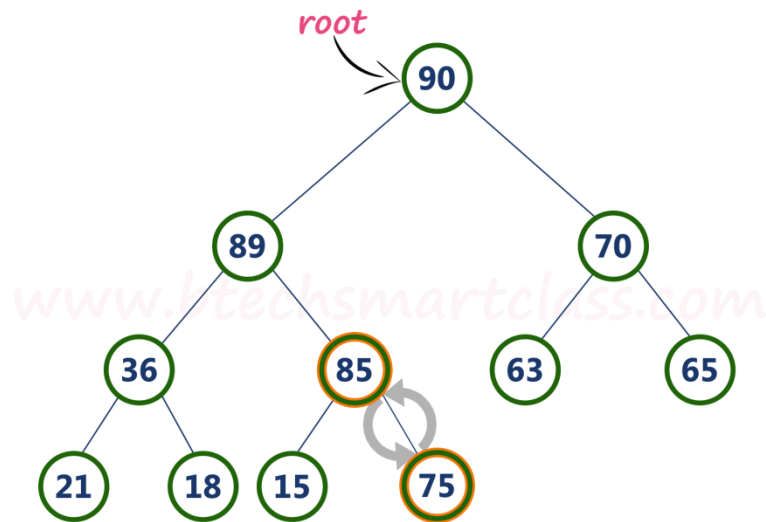
- **Step 1** - Insert the **newNode** with value 85 as **last leaf** from left to right. That means newNode is added as a right child of node with value 75. After adding max heap is as follows...



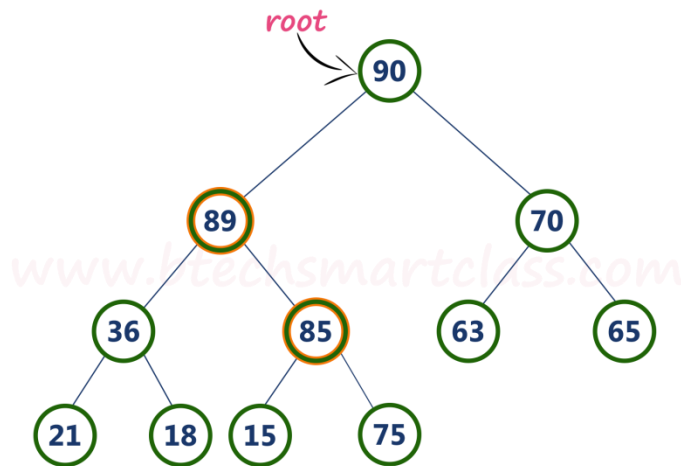
- **Step 2** - Compare **newNode value (85)** with its **Parent node value (75)**. That means **85 > 75**



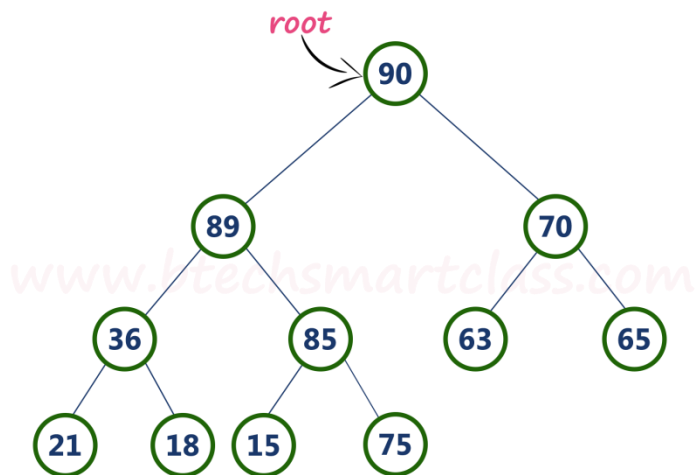
- **Step 3** - Here **newNode value (85)** is **greater** than its **parent value (75)**, then **swap** both of them. After swapping, max heap is as follows...



- **Step 4** - Now, again compare newNode value (85) with its parent node value (89).



Here, newNode value (85) is smaller than its parent node value (89). So, we stop insertion process. Finally, max heap after insertion of a new node with value 85 is as follows...



Deletion Operation in Max Heap

In a max heap, deleting the last node is very simple as it does not disturb max heap properties.

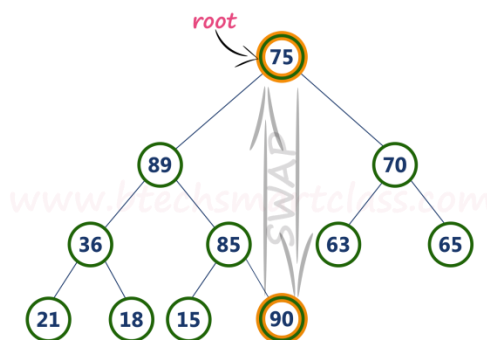
Deleting root node from a max heap is little difficult as it disturbs the max heap properties. We use the following steps to delete the root node from a max heap...

- **Step 1 - Swap** the **root** node with **last** node in max heap
- **Step 2 - Delete** last node.
- **Step 3 - Now**, compare **root value** with its **left child value**.
- **Step 4 - If root value is smaller** than its left child, then compare **left child** with its **right sibling**. Else goto **Step 6**
- **Step 5 - If left child value is larger** than its **right sibling**, then **swap root** with **left child** otherwise **swap root** with its **right child**.
- **Step 6 - If root value is larger** than its left child, then compare **root value** with its **right child** value.
- **Step 7 - If root value is smaller** than its **right child**, then **swap root** with **right child** otherwise **stop the process**.
- **Step 8 - Repeat** the same until root node fixes at its exact position.

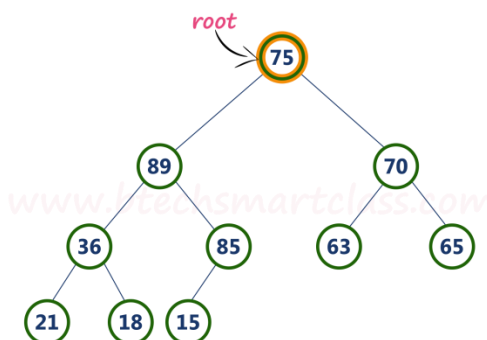
Example

Consider the above max heap. **Delete root node (90) from the max heap.**

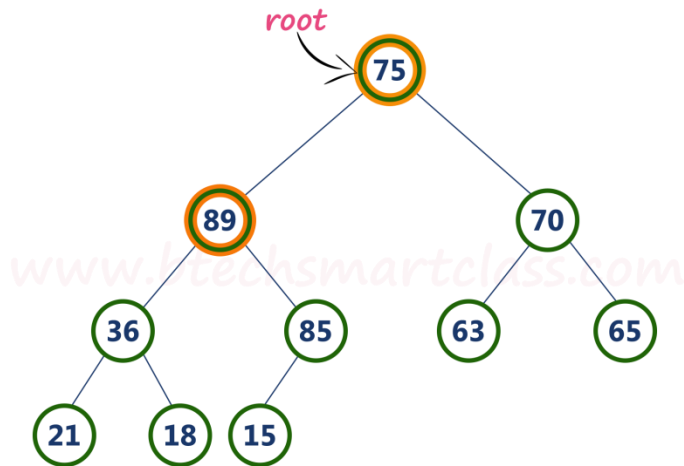
- **Step 1 - Swap** the **root node (90)** with **last node 75** in max heap. After swapping max heap is as follows...



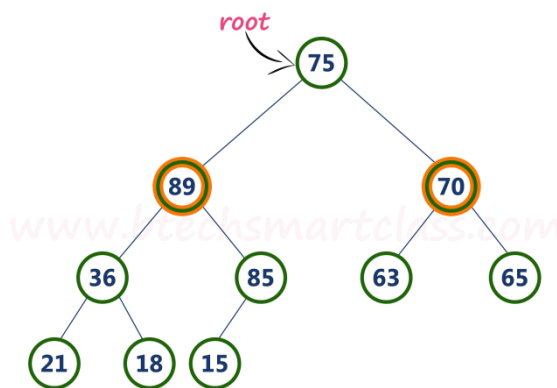
- **Step 2 - Delete** last node. Here the last node is 90. After deleting node with value 90 from heap, max heap is as follows...



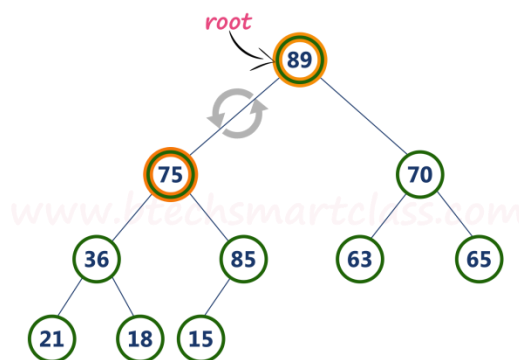
- **Step 3 - Compare** **root node (75)** with its **left child (89)**.



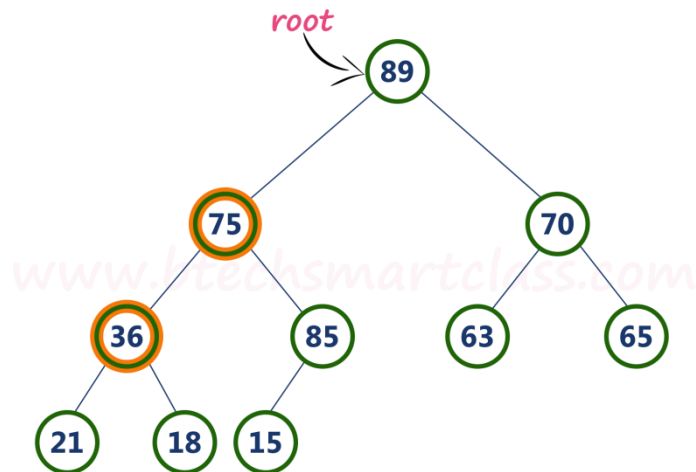
Here, **root value (75)** is **smaller** than its left child value (89). So, compare left child (89) with its right sibling (70).



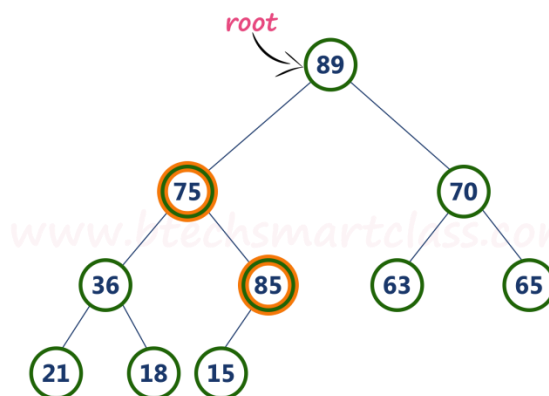
- **Step 4** - Here, **left child value (89)** is **larger** than its **right sibling (70)**, So, swap root (75) with left child (89).



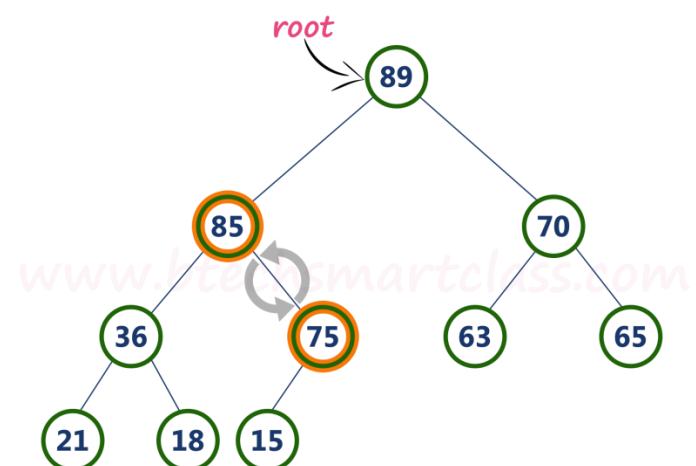
- **Step 5** - Now, again compare **75** with its **left child (36)**.



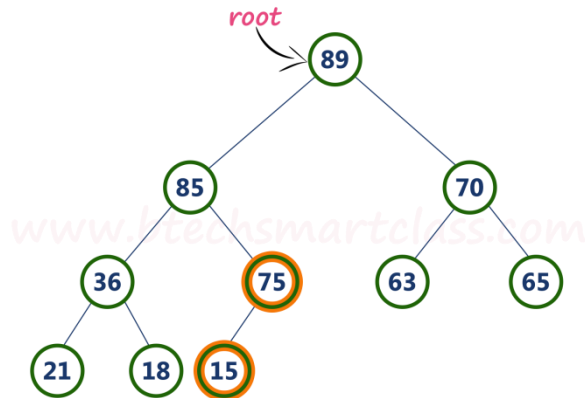
Here, node with value **75** is larger than its left child. So, we compare node **75** with its right child **85**.



- **Step 6** - Here, node with value **75** is smaller than its **right child (85)**. So, we swap both of them. After swapping max heap is as follows...

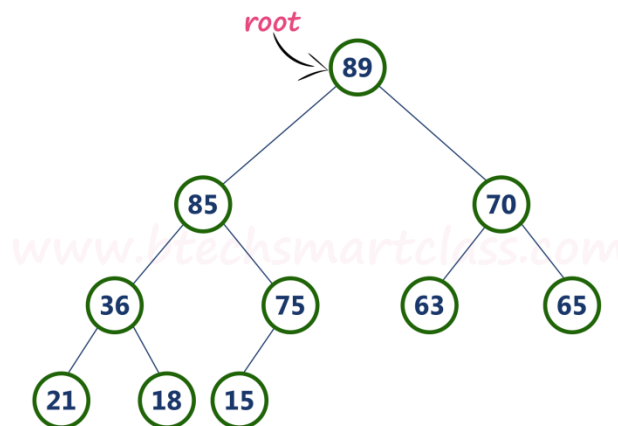


- **Step 7** - Now, compare node with value **75** with its left child (**15**).



Here, node with value **75** is larger than its left child (**15**) and it does not have right child. So we stop the process.

Finally, max heap after deleting root node (**90**) is as follows...

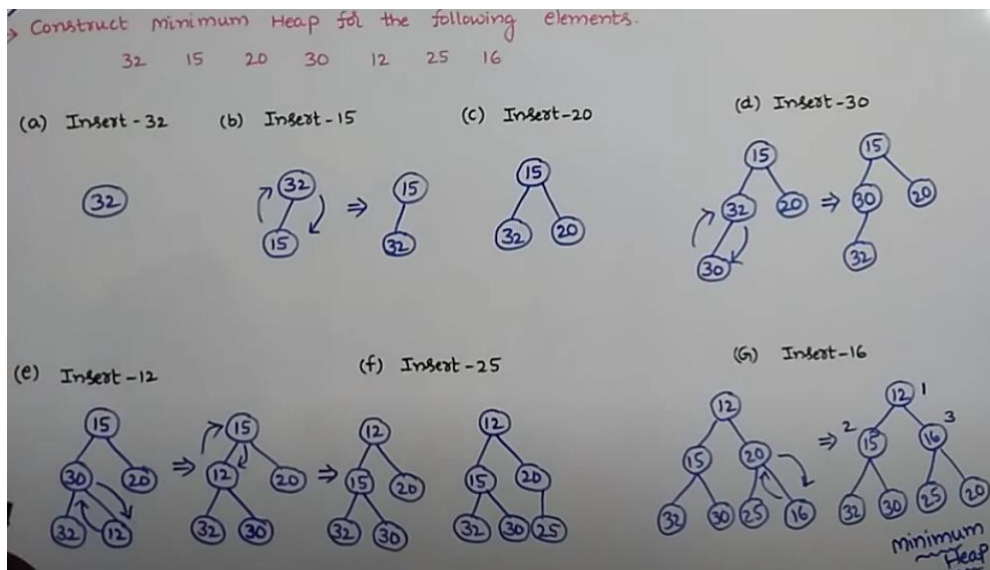


Min Heap:

Insertion Operation in Min Heap

Insertion Operation in min heap is performed as follows...

- **Step 1** - Insert the **newNode** as **last leaf** from left to right.
- **Step 2** - Compare **newNode value** with its **Parent node**.
- **Step 3** - If **newNode value is lesser** than its parent, then **swap** both of them.
- **Step 4** - Repeat step 2 and step 3 until newNode value is greater than its parent node (or) newNode reaches to root.

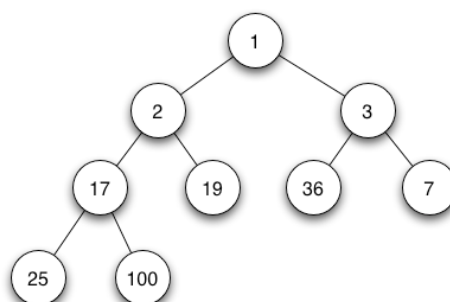


Deletion Operation in Min Heap

In a min heap, deleting the last node is very simple as it does not disturb max heap properties.

Deleting root node from a min heap is little difficult as it disturbs the min heap properties. We use the following steps to delete the root node from a min heap...

- **Step 1 - Swap** the **root** node with **last** node in min heap
- **Step 2 - Delete** last node.
- **Step 3 - Now**, compare **root value** with its **left child value**.
- **Step 4 - If root value is greater** than its left child, then compare **left child** with its **right sibling**. Else goto **Step 6**
- **Step 5 - If left child value is smaller** than its **right sibling**, then **swap root** with **left child** otherwise **swap root** with its **right child**.
- **Step 6 - If root value is smaller** than its left child, then compare **root value** with its **right child** value.
- **Step 7 - If root value is greater** than its **right child**, then **swap root** with **right child** otherwise **stop the process**.
- **Step 8 - Repeat** the same until root node fixes at its exact position.



Delete 100 and 25 from above tree..

Applications of Heap Trees:

Heap Data Structure is generally taught with Heapsort. Heapsort algorithm has limited uses because Quicksort is better in practice. Nevertheless, the Heap data structure itself is enormously used.

- **Priority Queues:** Heaps are commonly used to implement priority queues, where elements with higher priority are extracted first. This is useful in many applications such as scheduling tasks, handling interruptions, and processing events.
- **Sorting Algorithms:** Heapsort, a comparison-based sorting algorithm, is implemented using the Heap data structure. It has a time complexity of $O(n \log n)$, making it efficient for large datasets.
- **Graph algorithms:** Heaps are used in graph algorithms such as Prim's Algorithm, Dijkstra's algorithm., and the A* search algorithm.
- **Lossless File Compression:** Heaps are used in data compression algorithms such as Huffman coding, which uses a priority queue implemented as a min-heap to build a Huffman tree.
- **Medical Applications:** In medical applications, heaps are used to store and

manage patient information based on priority, such as vital signs, treatments, and test results.

- **Load balancing:** Heaps are used in load balancing algorithms to distribute tasks or requests to servers, by processing elements with the lowest load first.
- **Order statistics:** The Heap data structure can be used to efficiently find the kth smallest (or largest) element in an array. See method 4 and 6 of this post for details.
- **Resource allocation:** Heaps can be used to efficiently allocate resources in a system, such as memory blocks or CPU time, by assigning a priority to each resource and processing requests in order of priority.